TQL Design

– noch nicht abgestimmt –

TQL steht natürlich für *TAS Query Language*. Es handelt sich um eine *Domain Specific Language* (DSL) für die Definition von

- Wettkampf-Bewertungs- und Auswerte-Schemata (pro Rundenelement, Runde, Turnier)
- Bildungsregeln für Runden und Folge-Turniere (inkl. Klassifikationsrunden oder "Sieger tanzt mit")
- Regeln für die Ergebnisberichte (**TBD**: TQL-Nutzung sinnvoll?)

TAS-Interface

Zwei Tabellen mit vereinbarter Struktur bilden das Interface für die *Bewertung* für *beliebige* Wettkampfarten.
Design-Kriterium: Wenn sich sowohl das Skating-System als auch eine Fußball-WM damit modellieren lassen, sind wir für zukünftige Anforderungen aus dem Tanzsport gut aufgestellt.

Eingabe

- Tabelle **marks** (start, round, round_element, adjudicator_group, adjudicator, value DECIMAL)
- als *start* kann (muss aber nicht) die Startnummer verwendet werden

Ausgabe

- Tabelle **report** (guid, start, group-1 TEXT, ..., group-n TEXT, key TEXT, value DECIMAL, reason TEXT)
  - enhält das Endergebnis
  - enthält alle Daten, aus denen Report-Generatoren Auswertungs-Berichte erstellen sollen
  - *reason* dient optional zur Herstellung einer Traceability für den entsprechenden Wert (bzgl. Regeln und/oder verwendeter Eingabe-Daten)
- *report* enthält mindestens
  - einen ungruppierten key “*place-min*” pro Start, und
  - einen ungruppierten key “*place-max*” pro Start,
  - um das Endergebnis in Form eines Platz-von- bis Platz-bis-Bereichs darzustellen

Sprach-Features und Syntax

directive:

- DEFINE <identifier>(<param-list>) AS <macro-body>
- FUNCTION <identifier>[(<param-list>)]:
  - Basis-Typen (DECIMAL, STRING, NULL) werden als Wert übergeben
  - alle Tabellen-Objekte werden als Referenz übergeben
    - eine Übergabe als Wert kann die Funktion simulieren, indem der Parameter mit LET in eine neue Variable kopiert wird
- LET <identifier> = <expression>
- FOR <identifier> IN <limit>..<limit> {[ASC]|DESC}:
- FOR <identifier> IN <table>:
- FOR <identifier> IN <table>.<column>:
- FOR <identifier> IN (<value-list>):
- WHILE <condition>:
- BREAK
- CONTINUE
- WHEN <condition>:
- ELSE WHEN <condition>:
- ELSE:
- UPDATE <table> FROM <source-table> [USING {<source-column> [AS <target-column>] [,]}+]
  - nur Änderung von Werten
  - Wert der Ziel-Key-Spalte (obligatorisch) wählt die zu ändernden Zeilen
  - eine Quell-Zeile ohne Key-Match in der Ziel-Tabelle führt zum Abbruch
- CONCAT TO <target-table> FROM <source-table> [USING {<source-column> [AS <target-column>][,]}+]
  - nur neue Spalten
  - Wert der Ziel-Key-Spalte (obligatorisch) wählt die zu ändernden Zeilen
  - Hinweis: mit CONCAT TO können die meisten Use-Cases von JOIN in Standard-SQL gelöst werden
- CREATE <table-name> [IF NOT EXISTS]
- ADD COLUMN <column-name> {DECIMAL|TEXT|GUID} [DEFAULT <value>] TO TABLE <table> [IF NOT EXISTS]

- die erste Spalte einer Tabelle ist immer die einzige Primary-Key-Spalte
  - sie darf keine NULL-Werte enthalten
  - sie darf keine DEFAULT-Vereinbarung haben

table-expression:

- SELECT {*|<select-expression [AS <column-name>][,]}+ FROM <table>
  [WHERE <where-condition>]
  [ORDER BY <column-list> {[ASC]|DESC}]
  [GROUP BY <column-list> {[ASC]|DESC}]
  [LIMIT <numeric-expression> [ON DISTINCT <column-list>]]
  - liefert das Ergebnis als Tabellen-Objekt zurück
  - wenn die Key-Spalte der Tabelle (evtl. umbenannt) 1:1 zum Resultat gehört, dient sie als erste und Key-Spalte des Resultats
    - dies gilt auch, wenn sie im Resultat nicht ausdrücklich als erste Spalte aufgeführt ist
  - anderenfalls wird die erstgenannte Spalte des Resultats zur Key-Spalte
    - dazu muss natürlich der Key-Index neu aufgebaut werden
- UPDATE <target-table>
  [WHERE <where-condition>]
  SET <source-row> [USING {<source-column> [AS <target-column>][,]}+]
  - nur Änderung von Werten
  - *source-row* muss ein Tabellenausdruck mit genau einer Zeile sein
  - Wert der Ziel-Key-Spalte wird ignoriert, falls er in *source-row* vorkommt
  - wenn WHERE fehlt, werden alle Zeilen der Ziel-Tabelle geändert
  - liefert eine Tabellenspalte mit den Keys aller gematchten Zeilen der Ziel-Tabelle zurück (egal, ob die neuen Spaltenwerte sich von den alten unterscheiden)
  - darf auch als Direktive genutzt werden
- INSERT INTO <table> VALUES <table> [USING {<source-column> [AS <target-column>] [,]}+]
  - nur neue Zeilen
  - Key-Duplikation führt zum Abbruch
  - wenn der Key der Ziel-Tabelle vom Typ GUID ist, werden die neuen Keys automatisch erzeugt, sofern die Quell-Tabelle keine Daten für die Key-Spalte enthält
  - liefert eine Tabellenspalte mit allen neuen Keys in der Ziel-Tabelle zurück
  - darf auch als Direktive genutzt werden

- ({<column> = <expression>[,]}+)
  - liefert eine Tabellenzeile mit den aufgeführten Spaltenname-Wert-Paaren zurück
  - Datentypen werden automatisch übernommen
  - die erste Spalte ist die Key-Spalte (aus formalen Gründen)
  - DEFAULT für alle Spalten außer der Key-Spalte ist NULL
  - Hinweis: insbesondere nutzbar als *source-row* in UPDATE ... SET
- <table-expression>\[<index>\]
  - liefert die indizierte Tabellen-Zeile
  - Index 0 liefert die Spaltennamen
  - Index-Bereich für die Daten-Zeilen ist 1..n
- <table-expression>.<column-name>
  - liefert die ganze selektierte Spalte
  - stellt (aus formalen Gründen) die Key-Spalte des Resultats dar

expression:

- <table-expression>
  - wenn die resultierende Tabelle genau eine Spalte und eine Spalte enthält, kann dieser Ausdruck auch wie ein skalarer Wert verwendet werden
- NULL
- <numeric-expression>
- <string-pattern>
  - in “ “ eingeschlossen
  - Muster der Form {<expression>} werden durch den Wert des Ausdrucks ersetzt
  - geschweifte Klammern und doppelte Anführungszeichen müssen escaped werden: \{, \}, \”
- <string>
  - in ' ' eingeschlossen
  - einfache Anführungszeichen müssen escaped werden: \'
- unäre Operatoren: NOT, -, +, <value> IS [NOT] NULL
- binäre Operatoren: =, <>, <, <=, >, >=, +, -, *, /, %, AND, OR, XOR
- (<expression>)
- <function-name>(<param-list>)

select-expression (zusätzliche Terme):

- <column-name>

- <group-function>

where-condition (zusätzliche Terme):

- <column-name> IN <column-expression>
  - *column-expression* muss ein Tabellen-Ausdruck mit genau einer Spalte sein
    - wird *nicht* im Kontext der im SELECT verwendeten Tabelle ausgewertet
    - Spaltenname wird nicht beachtet

group-function:

- SUM(<select-expression>)
- COUNT({*|<select-expression>})
  - liefert mit * die Gesamtzahl aller Zeilen, die ggf. von WHERE ausgewählt wurden
  - sonst werden alle Werte des Ausdrucks gezählt, die nicht 0/''/NULL sind
- MIN(<select-expression>)
- MAX(<select-expression>)
- AVG(<select-expression>)
- FIRST(<select-expression>)
- LAST(<select-expression>)
- MEDIAN(<select-expression>)

Lexikalische Struktur

- Blöcke für Kontrollstrukturen (FUNCTION, FOR, WHEN, WHILE) werden durch Einrückung definiert (analog Python)
- Direktiven enden **nicht** mit einem Semikolon!
- Text zwischen # und dem Zeilenende ist ein Kommentar

Tabellen und Indizierung

- TQL arbeitet nur auf internen Tabellen-Objekten:
  - Die Anbindung an die Backend-Datenbank erfolgt *nur* vor und *nach* der Ausführung ein oder mehrerer TQL-Skripte, nicht *während* dieser Ausführung.
  - Dadurch wird die Backend-Datenbank nicht inkonsistent, wenn TQL durch einen Fehler abbricht (implizite **Transaktions-Semantik**).
  - Die internen Tabellen können erzeugt werden
    - von der Umgebung vor dem Skript-Aufruf, und/oder
    - vom TQL-Skript via *table-expression*.

- Eine Tabelle mit mindestens einer Spalte verwendet immer die erste Spalte als eindeutigen Primärschlüssel (a.k.a. PRIMARY-KEY aus SQL). Über den Key werden
  - die Zeilen-Assoziationen in den Direktiven UPDATE ... FROM und CONCAT TO ... FROM gesteuert, und
  - die Auswertung von WHERE-Bedingungen optimiert.

Implementierungsvorschlag für Tabellen-Objekte

```
Table = (
    column_list : SortedDict<column_name : String,   column_index : Int             > ;
    row_order   : SortedDict<row_index   : Int,      key_value    : TqlValue        > ;
    rows        : SortedDict<key_value   : TqlValue, row          : Vector<TqlValue>> ;
    defaults    : Vector<TqlValue>                                                    ;
)
```

Rationale

- *column_list* erlaubt schnellen Zugriff auf Spalten per Name
  - Enumeration nach Spalten-Index wird kaum benötigt (nur Debugging + Ausgabe)
- Zugriffe auf Zeilen via Key nutzen *rows* und sind am schnellsten
  - häufigste Art des Zugriffs
- die optionale Sortierung von Zeilen unabhängig von der Key-Spalte wird separat in *row_order* abgelegt
  - nur für SELECT ... ORDER BY technisch erforderlich
- Zeilen sind als Vector modelliert:
  - schließt auch die Werte der Key-Spalte ein – technisch redundant, aber spart Fallunterscheidungen in der Implementierung
  - Spalten werden kaum jemals zugefügt ⇒ höherer Aufwand ok
  - Vektoren können ggf. mit Reserve-Kapazität angelegt werden, da sie vermutlich nur Referenzen auf *TqlValue*-Objekte enthalten
- Ablage der DEFAULT-Werte in *defaults* ist kompatibel zum Ablageformat der Zeilen ⇒ Initialisierung per Zuweisung möglich